2026 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
September 27 - October 1, 2026. Pittsburgh, PA, USA

FDC-MPPI: Fast Disturbance-Compensated MPPI for Onboard Quadruped Navigation in Cluttered and Disturbed Environments

Jialang Cai[1,*], Jiaheng Lu[1,*], Jinya Su[1], *Senior Member, IEEE*, Lingying Huang[1], Shihua Li[1], *IEEE Fellow*

***Abstract*— Quadruped robots struggle with reliable navigation in cluttered, narrow and disturbed environments because robot system and terrain-induced disturbances (e.g., slippage, traction loss, external pushes) cause a time-varying mismatch between navigation model and real system, which degrades Model Predictive Path Integral (MPPI) rollouts and leads to oscillations or collisions. This paper presents FDC-MPPI (Fast Disturbance-Compensated MPPI), an integrated planning-control framework that explicitly fuses real-time odometry feedback into the MPPI rollout model via a Fast and Robust Moving Horizon Estimation (FR-MHE) module. FR-MHE estimates disturbance compensation coefficients from the equivalent control input and measured robot pose, maps them to body-frame gains, and embeds them into the MPPI model to correct rollouts online while obstacle costs are efficiently queried from an Euclidean Distance Transform-based distance field. With GPU parallelization and a decoupled control-sampling architecture, FDC-MPPI achieves a 150 Hz onboard update rate on an NVIDIA Jetson Orin NX 16GB (average latency 2.42 ms, FR-MHE 0.84 ms). Comparative Gazebo simulations and real-world experiments on a Unitree GO2 demonstrate substantially improved robustness and safety across slippery slopes, outdoor lawn, and external forces, boosting success rates from 10%/25% (sim/real) for the Vanilla MPPI to 100%/90%, while increasing obstacle clearance and reducing trajectory tracking error and curvature—enabling agile, stable, and collision-resistant quadrupedal navigation under severe model mismatch and disturbances. GitHub page with videos is available at `https://langcode-hub.github.io/FDC-MPPI/`.**

I. INTRODUCTION

As high-inertia legged systems, quadruped platforms inherently suffer from delayed and imperfect responses to velocity commands [1], [2], a challenge further compounded by unknown disturbances induced by rough and unstructured terrains. Such time-varying uncertainties often lead to a pronounced sim-to-real gap [3] for learning community or model mismatch for control community [4]: even minor model or estimation errors can rapidly accumulate and cause catastrophic mission failures in safety-critical applications [5]. Model Predictive Path Integral (MPPI) has emerged as a promising framework for gradient-free optimization control under complex and non-smooth costs [6], [7], standard schemes typically adopt simplified models for fast rollout and fixed noise assumptions, which fail to capture environmental perturbations and low-level execution errors [8], [9], [10].

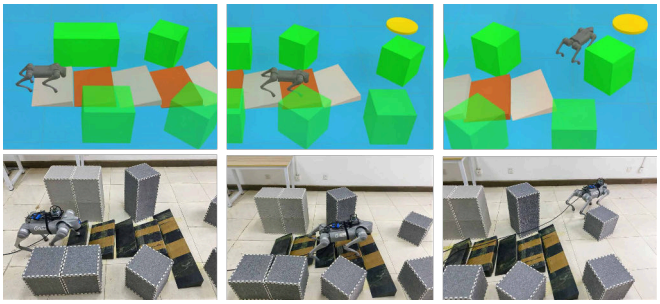


Fig. 1. **Video: FDC-MPPI navigation in cluttered environments**: validated in both Gazebo simulator and on a quadruped robot.

To bridge this gap, we present *Fast Disturbance-Compensated MPPI (FDC-MPPI)*, a fast and robust onboard navigation framework for cluttered and disturbed environments. Inspired by [11], FDC-MPPI is to transform adverse environmental effects into actionable compensation via better MPPI rollout modeling. To this end, we leverage *Fast and Robust Moving Horizon Estimation (FR-MHE)* [12] into the MPPI framework to achieve high-precision, real-time estimation of external perturbations and velocity deviations from odometry feedback. These disturbances are mapped into real-time disturbance compensation coefficients that dynamically augment the MPPI rollout model, improving rollout fidelity and the reliability of cost evaluation. By utilizing custom CUDA kernels for hardware acceleration and decoupling control frequency from sampling intervals [6], FDC-MPPI achieves a high-frequency (150 Hz) update rate using only limited onboard computational resources.

(1) **FDC-MPPI framework:** We propose FDC-MPPI, which enhances quadrupedal navigation stability and safety by explicitly integrating a real-time disturbance compensation mechanism into MPPI rollouts.
(2) **FR-MHE for disturbance compensation:** We integrate a computationally efficient and robust FR-MHE module with MPPI to actively mitigate environmental perturbations and velocity fluctuations.
(3) **Onboard validation:** Extensive simulations and hardware experiments verify algorithm robustness and efficiency (2.42ms MPPI computation and 150 Hz) in narrow, cluttered, and disturbed environments.

*These authors contributed equally to this work. This work was supported by National Natural Science Foundation of China under Grant 62303110, the National Industrial Internet Industry-Education Integration Platform, Southeast University, Basic Research Program of Jiangsu under Grant No. BK20253020, Shenzhen Science and Technology Program under Grant KJZD20240903100814018, the Start-Up Research Fund of Southeast University under Grant RF1028623226. Corresponding author: Jinya Su.

[1]All authors are with School of Automation, Key Laboratory of Measurement and Control of CSE, Ministry of Education, and Institute of Intelligent Unmanned Systems, Southeast University, Nanjing 210096, China. Emails: jalen.robotics@gmail.com; superjhlu@seu.edu.cn; sucas@seu.edu.cn; lingyinghuang@seu.edu.cn; lsh@seu.edu.cn.

Copyright ©2026 IEEE

(4) **Open-source code repository:** We will provide an open-source implementation of the MPPI-based local obstacle avoidance algorithm for quadruped robots.

II. RELATED WORK

A. MPPI in Robotics

Compared with the classical Model Predictive Control (MPC) [5], a commonly used approach for predictive control under constraints and differentiable costs, MPPI follows a sampling-based optimization paradigm that avoids explicit gradient computation of the cost. Complex constraints can also be naturally incorporated into the objective as soft penalties [13], and modern GPU massive parallelism enables efficient evaluation of thousands of rollouts per control cycle. MPPI has been widely adopted in UAVs control [7], robotic manipulation [14], and quadrupedal locomotion [15], [16]. Most existing systems execute MPPI on high-performance desktop workstations and stream control commands over local networks, which limits robot mobility, increases latency sensitivity, and overlooks the practical requirement of sustaining high MPPI update rates on resource-constrained embedded platforms for fully onboard autonomy.

B. Robot Local Navigation

SLAM-based navigation methods typically rely on prior maps and employ a hierarchical planning–control architecture [17], which can introduce mismatches between high-level path generation and low-level dynamic feasibility, especially in high-speed or cluttered environments. In contrast, as an integrated planning–control scheme, MPPI can perform local obstacle avoidance using only a goal specification and without requiring prior environmental knowledge. Although several studies have applied MPC or MPPI to robot local navigation, many do not explicitly address severe model mismatches and robot-terrain disturbances; moreover, limited onboard update frequencies reduce the ability to react to fast-changing hazards in confined spaces.

Learning-based navigation approaches have also gained popularity due to their ability in handling sparse rewards and non-differentiable costs, yet it often suffers from low sample efficiency and limited generalization to different platforms or tasks [18], making online adaptation to unexpected disturbances difficult [19], [20]. These challenges are amplified for high-dimensional, under-actuated quadrupedal platforms, where strong disturbances (e.g., slippery slopes in Fig. 1) can induce abrupt velocity fluctuations that directly corrupt the predictive ability of the MPPI rollout model.

C. Disturbance Compensation

Integrating disturbance estimation for better model based control is practically significant [7]. In this regard, accurate and low-latency disturbance estimation and compensation is critical [21]. While conventional filtering methods (e.g., EKF, UKF) and learning-based estimators can offer computational efficiency or modeling flexibility, they often struggle under strong nonlinearities, non-Gaussian noise, or stringent onboard resource constraints. Optimization-based MHE provides improved robustness when handling constraints and nonlinear dynamics [22], [23], [24].

However, standard MHE typically requires iterative optimization, which can be difficult to reconcile with the real-time demands of high-frequency MPPI rollouts on embedded hardware. To bridge this gap, we leverage *Fast and Robust MHE (FR-MHE)*, a single-iteration gradient-based estimator that converts velocity perturbations into actionable compensation coefficients with minimal latency.

Therefore, we propose FDC-MPPI, which integrates FR-MHE to balance state and disturbance estimation accuracy and computational efficiency, mapping velocity perturbations induced by external forces or low-level execution errors into disturbance compensation coefficients that augment the MPPI rollout model. Leveraging an Jetson Orin NX 16GB, our framework sustains a sub-7 ms control cycle, providing the rapid reflex capability for collision-free navigation in narrow, cluttered and disturbed environments.

III. FDC-MPPI FRAMEWORK

A. MPPI for Local Navigation

MPPI offers an alternative for solving complex, non-convex planning-control problems by using Monte Carlo sampling to explore the control space. Rather than relying on gradient-based optimization [5], [25], MPPI evaluates a large number of rollouts in parallel and directly selects control updates according to their task performance. This sampling-based formulation enables the seamless integration of high-fidelity system dynamics and rich environmental constraints through cost shaping, avoiding restrictive approximations that are often required for tractable analytical optimization.

At each time step $k \in \{0, \dots, K-1\}$, starting from the current state, MPPI generates N candidate control sequences by perturbing a nominal control sequence $\boldsymbol{u}^{\text{nom}}$ with Gaussian noise $\boldsymbol{\delta}_k^n \sim \mathcal{N}(0, \Sigma)$, where $n \in \{0, \dots, N-1\}$. Each perturbed control input $\boldsymbol{u}_k^n = \boldsymbol{u}_k^{\text{nom}} + \boldsymbol{\delta}_k^n$ is used to simulate a forward trajectory across the prediction horizon K using the rollout model. In this study, without loss of generality, the following kinematic navigation model [11] is considered.

$$ {}_{\mathcal{B}}\boldsymbol{x}_{k+1} = {}_{\mathcal{B}}\boldsymbol{x}_k + \boldsymbol{R}_{t,k}({}_{\mathcal{B}}\theta) \cdot \boldsymbol{u}_k \cdot T \tag{1} $$

where T is the discrete time-step length for rollout simulation and ${}_{\mathcal{B}}\boldsymbol{x} = [{}_{\mathcal{B}}p_x, {}_{\mathcal{B}}p_y, {}_{\mathcal{B}}\theta]^T$ denotes the simulated pose vector in the body frame $\{\mathcal{B}\}$, representing the robot's local position and heading relative to its own orientation. The control sequence sampled by MPPI is $\boldsymbol{u} = [{}_{\mathcal{B}}v_x, 0, \omega]^T$ (i.e., velocity and angular velocity in the body coordinate), and $\boldsymbol{R}_t({}_{\mathcal{B}}\theta)$ is the rotation matrix associated with the simulated heading ${}_{\mathcal{B}}\theta$, used to transform sampled commands into the body-frame velocity vector under the robot's coordinate convention.

After computing the rollouts, each trajectory is evaluated by a task-specific cost function $J_n = \Phi_n(K) + \sum_{k=0}^{K-1} L_n({}_{\mathcal{B}}\boldsymbol{x}_k, \boldsymbol{u}_k)$. The stage cost L_n consists of three

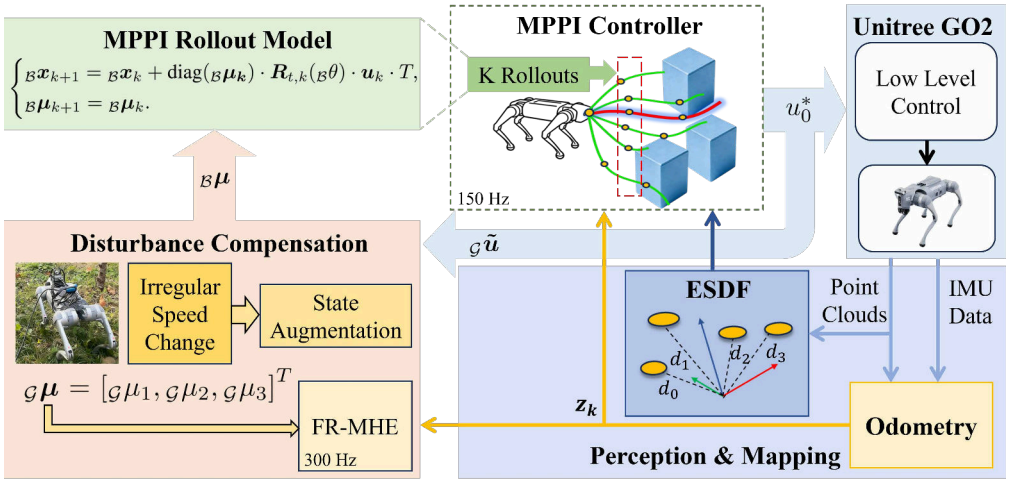


Fig. 2. **Framework of FDC-MPPI.** It first uses FR-MHE to estimate global-frame disturbance compensation coefficients ${}_{\mathcal{G}}\boldsymbol{\mu}$ by fusing odometer-derived robot poses with the equivalent control input ${}_{\mathcal{G}}\tilde{\boldsymbol{u}}$. These coefficients are mapped to body-frame gains ${}_{\mathcal{B}}\boldsymbol{\mu}$ and embedded into the MPPI rollout to actively rectify trajectory predictions against environmental perturbations. Finally, the MPPI evaluates N rollouts and calculates the optimal body-frame control input u_0^* through a weighted average, which is executed by the low-level controller.

terms, $L_n = l_{\text{direction}} + l_{\text{collision}} + l_{\text{progress}}$, defined as

$$\begin{cases} l_{\text{direction}} = W_{\text{direction}} \cdot \left\| \left({}_{\mathcal{G}}\psi_{\text{target}} - \left({}_{\mathcal{G}}\psi_{\text{body}} + {}_{\mathcal{B}}\theta_k \right) \right) \right\|^2 \\ l_{\text{collision}} = W_{\text{collision}} \cdot \left(\dfrac{1}{d_{\text{collision}}(k) + \epsilon} \right)^2_{\left\{ d_{\text{collision}} \le d_{\text{safe}} \right\}} \\ l_{\text{progress}} = W_{\text{progress}} \cdot min \left\{ 0, d_{\text{target}}\left(k+1\right) - d_{\text{target}}(k) \right\} \end{cases} \tag{2}$$

where ${}_{\mathcal{G}}\psi_{\text{target}}$ and ${}_{\mathcal{G}}\psi_{\text{body}}$ denote the target heading and the robot body heading in the global frame $\{\mathcal{G}\}$. $d_{\text{collision}}$ and d_{target} represent the distances from a sampled trajectory point to the nearest obstacle and to the target.

1) $l_{direction}$: To penalize heading deviation during motion, this term measures the squared error between the desired heading ${}_{\mathcal{G}}\psi_{\text{target}}$ and the simulated robot heading $({}_{\mathcal{G}}\psi_{\text{body}} + {}_{\mathcal{B}}\theta)$ along the rollout.

2) $l_{collision}$: To avoid obstacles, MPPI computes $d_{\text{collision}}$ from the Euclidean Signed Distance Field (ESDF) map [26]. When sampled points fall within the safe distance d_{safe}, $l_{\text{collision}}$ penalizes them sharply to steer rollouts away from obstacles.

3) $l_{progress}$: Following [27], l_{progress} measures progress toward the target between two successive rollout states. A negative value indicates the robot is moving closer to the target and thus acts as a reward.

The terminal cost Φ_n imposes a stronger constraint on the final predicted state to encourage convergence:

$$\begin{aligned} \Phi_n = & - W_{\text{progress}} \cdot max \left\{ 0, d_{\text{target}}\left(0\right) - d_{\text{target}}\left(K-1\right) \right\} \\ & + W_{\text{term,residual}} \cdot d_{\text{target}}\left(K-1\right) \\ & + W_{\text{term,direction}} \cdot \left\| {}_{\mathcal{G}}\psi_{\text{target}} - \left({}_{\mathcal{G}}\psi_{\text{body}} + \theta_{K-1} \right) \right\|^2 \end{aligned} \tag{3}$$

After calculating the navigation cost J_n for each rollout, MPPI assigns a normalized importance weight

$$\varepsilon_n = \frac{\exp\left(-\frac{1}{\lambda}\left(J_n - J_{\text{min}}\right)\right)}{\sum_{n=0}^{N-1} \exp\left(-\frac{1}{\lambda}\left(J_n - J_{\text{min}}\right)\right)} \tag{4}$$

where J_{min} is the minimum rollout cost among the N samples and λ is a temperature parameter controlling the concentration of weights. The optimal control sequence is updated by the weighted average

$$\boldsymbol{u}^* = \boldsymbol{u}^{\text{nom}} + \sum_{n=0}^{N-1} \varepsilon_n \delta \boldsymbol{u}^n. \tag{5}$$

B. FR-MHE for Disturbance Coefficient Estimation

To mitigate the adverse effects of lumped disturbances, we introduce disturbance estimation module for reliable MPPI rollout and cost calculation. As shown in TABLE I (note: "R"–"Robust", means explicitly considering disturbances), existing estimation approaches face a fundamental trade-off: filters (e.g., EKF variants) are lightweight but often fragile under strong nonlinearities and non-Gaussian disturbances, whereas optimization-based MHE/RMHE is robust yet typically too computationally expensive for high-rate onboard control. To close this gap, we leverage FR-MHE by simplifying the MHE computation while explicitly handling lumped disturbances by augmenting and estimating disturbance-related variables.

For planar motion, we first map the original MPPI control input $\boldsymbol{u}$ equivalently into the global frame $\{\mathcal{G}\}$. Denoting ${}_{\mathcal{G}}\tilde{\boldsymbol{u}} = [{}_{\mathcal{G}}\tilde{v}_x, {}_{\mathcal{G}}\tilde{v}_y, \tilde{\omega}]^T$, we have

$$\begin{aligned} {}_{\mathcal{G}}\tilde{\boldsymbol{u}} &= {}^{\mathcal{G}}\boldsymbol{R}_{\mathcal{B}}({}_{\mathcal{G}}\psi_{\text{body}}) \cdot \boldsymbol{u} \\ {}^{\mathcal{G}}\boldsymbol{R}_{\mathcal{B}}({}_{\mathcal{G}}\psi_{\text{body}}) &= \begin{bmatrix} \cos({}_{\mathcal{G}}\psi_{\text{body}}) & -\sin({}_{\mathcal{G}}\psi_{\text{body}}) & 0 \\ \sin({}_{\mathcal{G}}\psi_{\text{body}}) & \cos({}_{\mathcal{G}}\psi_{\text{body}}) & 0 \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned} \tag{6}$$

As shown in Eq. (7), we model disturbances in the control system as proportional compensation coefficients ${}_{\mathcal{G}}\boldsymbol{\mu} = [{}_{\mathcal{G}}\mu_1, {}_{\mathcal{G}}\mu_2, {}_{\mathcal{G}}\mu_3]^T$ acting on the equivalent control input. Assuming these coefficients are slowly time-varying (approximately constant) over a short interval, we augment the robot pose state in $\{\mathcal{G}\}$ with ${}_{\mathcal{G}}\boldsymbol{\mu}$ and construct the discrete-time

TABLE I

COMPARISON OF STATE ESTIMATION ALGORITHMS

Methods	Robustness	Implementation	Real-time	Non-Gaussian
EKF	**Low**	**Low**	✓	×
REKF	**Medium**	**Medium**	✓	×
MHE	**High**	**High**	×	✓
RMHE	**High**	**High**	×	✓
FR-MHE (ours)	**High**	**Medium**	✓	✓

augmented state vector $\boldsymbol{\xi} = [{}_{\mathcal{G}}p_x, {}_{\mathcal{G}}p_y, {}_{\mathcal{G}}\theta, {}_{\mathcal{G}}\mu_1, {}_{\mathcal{G}}\mu_2, {}_{\mathcal{G}}\mu_3]^T$. The corresponding discrete-time process dynamics $\boldsymbol{\xi}_{k+1} = \boldsymbol{f}_{\text{aug}}(\boldsymbol{\xi}_k, {}_{\mathcal{G}}\tilde{\boldsymbol{u}})$ is expressed as

$$\underbrace{\begin{bmatrix} {}_{\mathcal{G}}\boldsymbol{x}_{k+1} \\ {}_{\mathcal{G}}\boldsymbol{\mu}_{k+1} \end{bmatrix}}_{\boldsymbol{\xi}_{k+1}} = \underbrace{\begin{bmatrix} {}_{\mathcal{G}}\boldsymbol{x}_k + \text{diag}({}_{\mathcal{G}}\boldsymbol{\mu}_k) \cdot {}_{\mathcal{G}}\tilde{\boldsymbol{u}} \cdot T \\ {}_{\mathcal{G}}\boldsymbol{\mu}_k \end{bmatrix}}_{\boldsymbol{f}_{\text{aug}}(\boldsymbol{\xi}_k, {}_{\mathcal{G}}\tilde{\boldsymbol{u}})}, \tag{7}$$

where the measurement equation relies on odometry data in global coordinate $\boldsymbol{z}_k = [{}_{\mathcal{G}}p_{x,\text{odom}}, {}_{\mathcal{G}}p_{y,\text{odom}}, {}_{\mathcal{G}}\psi_{\text{body}}]^T$.

In this system, FR-MHE [12] maintains a moving observation window of length $M = 8$ to fuse time-series measurements and handle nonlinearities. The goal is to compute the current optimal estimate $\hat{\boldsymbol{\xi}}_k$ by minimizing the windowed cost function S_k, which includes (i) a running cost from measurement residuals within the window and (ii) an arrival cost encoding historical prior information:

$$S_k(\hat{\boldsymbol{\xi}}_k) = \underbrace{\sum_{k=t}^{t+M-1} W_{\text{pos}} \left\| \boldsymbol{H}_k \boldsymbol{\xi}_k - \boldsymbol{z}_k \right\|^2}_{\text{Running Cost}} + \underbrace{W_{\text{prior}} \left\| \hat{\boldsymbol{\xi}}_k - \bar{\boldsymbol{\xi}}_k \right\|^2}_{\text{Arrival Cost}}, \tag{8}$$

where $\boldsymbol{H}_k \in \mathbb{R}^{3\times 6}$ is the observation matrix that extracts the pose variables, W_{pos} and W_{prior} are the measurement and prior weights, and $\bar{\boldsymbol{\xi}}_k$ denotes the prior state obtained by forward propagation of (7) from the previous optimal estimate.

Augmenting ${}_{\mathcal{G}}\boldsymbol{\mu}$ increases dimension and can impose additional computation that compromises MPPI's real-time control frequency. To achieve ultra-low latency, FR-MHE adopts a *single-iteration* gradient-descent update evaluated at the prior state, which simplifies the arrival-cost contribution:

$$\hat{\boldsymbol{\xi}}_k = \bar{\boldsymbol{\xi}}_k - \alpha \nabla \phi_k(\bar{\boldsymbol{\xi}}_k), \tag{9}$$

where α is the diagonal step-size matrix. Since the gradient is evaluated at the prior $\bar{\boldsymbol{\xi}}_k$, the partial derivative of the arrival-cost term at this point is zero:

$$\nabla \left(W_{\text{prior}} \left\| \hat{\boldsymbol{\xi}}_k - \bar{\boldsymbol{\xi}}_k \right\|^2 \right) \Big|_{\hat{\boldsymbol{\xi}}_k = \bar{\boldsymbol{\xi}}_k} = 0. \tag{10}$$

Thus, when computing the total gradient $\nabla \phi_k(\bar{\boldsymbol{\xi}}_k)$, FR-MHE only requires the gradient of the measurement residual term. Using the chain rule, we analytically derive the Jacobian of the composite process equation, with the forward recursion

$$\boldsymbol{DF}_{k+1} = \boldsymbol{J}_f(\boldsymbol{\xi}_k, {}_{\mathcal{G}}\tilde{\boldsymbol{u}})\, \boldsymbol{DF}_k, \tag{11}$$

Algorithm 1 FR-MHE for Coefficient Estimation

Require: Measurements $\boldsymbol{z}_{t:k}$, commands $\boldsymbol{u}_{t:k}$, headings ${}_{\mathcal{G}}\psi_{\text{body},t:k}$, window M $(t = k - M + 1)$, prior $\bar{\boldsymbol{\xi}}_t$, step $\boldsymbol{\alpha}$, weight W_{pos}.

Ensure: ${}_{\mathcal{G}}\hat{\boldsymbol{\mu}}_k$, updated $\hat{\boldsymbol{\xi}}_k$, next prior $\bar{\boldsymbol{\xi}}_{k+1}$.

1: **Global-input transform:** ${}_{\mathcal{G}}\tilde{\boldsymbol{u}}_i \leftarrow \boldsymbol{R}^{\mathcal{G}}_{\mathcal{B}}({}_{\mathcal{G}}\psi_{\text{body},i})\, \boldsymbol{u}_i,\ \forall i = t, \ldots, k$

2: **Prior rollout:** $\bar{\boldsymbol{\xi}}_{i+1} \leftarrow f_{\text{aug}}(\bar{\boldsymbol{\xi}}_i, {}_{\mathcal{G}}\tilde{\boldsymbol{u}}_i),\ \forall i = t, \ldots, k-1$

3: **Residuals:** $\boldsymbol{r}_i \leftarrow \boldsymbol{H}\bar{\boldsymbol{\xi}}_i - \boldsymbol{z}_i,\ \forall i = t, \ldots, k$

4: **One-shot gradient:**

5: $\boldsymbol{g} \leftarrow \text{FRGRAD}(\{\bar{\boldsymbol{\xi}}_i\}, \{{}_{\mathcal{G}}\tilde{\boldsymbol{u}}_i\}, \{\boldsymbol{r}_i\}, \boldsymbol{H}, W_{\text{pos}})$

6: **Single-step update:** $\hat{\boldsymbol{\xi}}_t \leftarrow \bar{\boldsymbol{\xi}}_t - \boldsymbol{\alpha}\boldsymbol{g}$

7: **Re-rollout:** $\hat{\boldsymbol{\xi}}_{i+1} \leftarrow f_{\text{aug}}(\hat{\boldsymbol{\xi}}_i, {}_{\mathcal{G}}\tilde{\boldsymbol{u}}_i),\ \forall i = t, \ldots, k-1$

8: **Extract:** ${}_{\mathcal{G}}\hat{\boldsymbol{\mu}}_k \leftarrow \hat{\boldsymbol{\xi}}_k$ ▷ disturbance coeff. components

9: **Shift prior:** $\bar{\boldsymbol{\xi}}_{k+1} \leftarrow f_{\text{aug}}(\hat{\boldsymbol{\xi}}_k, {}_{\mathcal{G}}\tilde{\boldsymbol{u}}_k)$

where $\boldsymbol{DF}_k = \frac{\partial \boldsymbol{\xi}_k}{\partial \boldsymbol{\xi}_t}$ denotes the composite Jacobian of the predicted state at step k with respect to the window-start state, initialized as $\boldsymbol{DF}_t = \boldsymbol{I}$, and $\boldsymbol{J}_f(\boldsymbol{\xi}_k, {}_{\mathcal{G}}\tilde{\boldsymbol{u}}) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{\xi}}\big|_{(\boldsymbol{\xi}_k, {}_{\mathcal{G}}\tilde{\boldsymbol{u}})}$ is the local Jacobian of $\boldsymbol{f}_{\text{aug}}$ at the current operating point.

This simplification avoids Hessian computation and nonlinear solvers. Instead, FR-MHE achieves efficient and accurate estimation of disturbance compensation coefficients by tuning α across state variables, implicitly balancing prior confidence and measurement corrections (e.g., using larger step sizes for $\boldsymbol{\mu}$ to accelerate adaptation under sudden perturbations). The steps of the FR-MHE are summarized in Algorithm 1.

C. Rollout Model and Implementation

After estimating the disturbance compensation coefficients ${}_{\mathcal{G}}\boldsymbol{\mu}$ via FR-MHE, we compensate the equivalent control input in the global frame $\{\mathcal{G}\}$ as

$$\begin{bmatrix} {}_{\mathcal{G}}\tilde{v}_x^{\text{c}} \\ {}_{\mathcal{G}}\tilde{v}_y^{\text{c}} \\ \tilde{\omega}^{\text{c}} \end{bmatrix} = \begin{bmatrix} {}_{\mathcal{G}}\mu_1 & 0 & 0 \\ 0 & {}_{\mathcal{G}}\mu_2 & 0 \\ 0 & 0 & {}_{\mathcal{G}}\mu_3 \end{bmatrix} \begin{bmatrix} {}_{\mathcal{G}}\tilde{v}_x \\ {}_{\mathcal{G}}\tilde{v}_y \\ \tilde{\omega} \end{bmatrix}, \tag{12}$$

where the superscript "c" denotes the control quantity after disturbance compensation.

However, MPPI commands are applied in the body frame $\{\mathcal{B}\}$. We therefore seek body-frame disturbance coefficients ${}_{\mathcal{B}}\boldsymbol{\mu} = [{}_{\mathcal{B}}\mu_1, 0, {}_{\mathcal{B}}\mu_2]^T$. Using the coordinate transformation in (6) and match Eq. (12), we link the compensated global-frame command to the body-frame MPPI control variables.

$$\begin{cases} {}_{\mathcal{G}}\tilde{v}_x^{\text{c}} = {}_{\mathcal{G}}\mu_1 \cdot {}_{\mathcal{B}}v_x \cos({}_{\mathcal{G}}\psi_{\text{body}}), \\ {}_{\mathcal{G}}\tilde{v}_y^{\text{c}} = {}_{\mathcal{G}}\mu_2 \cdot {}_{\mathcal{B}}v_x \sin({}_{\mathcal{G}}\psi_{\text{body}}), \\ \tilde{\omega}^{\text{c}} = {}_{\mathcal{G}}\mu_3 \cdot \omega, \end{cases} \tag{13}$$

Next, we rotate the quantity on the left side of Eq. (13) into the compensated control quantity in $\{\mathcal{B}\}$ to derive ${}_{\mathcal{B}}\boldsymbol{\mu}$. Since the MPPI control variables do not include lateral velocity, ${}_{\mathcal{B}}v_y^{\text{c}}$ is omitted in (13). The resulting compensation coefficients for the MPPI control variables are

$$\begin{cases} {}_{\mathcal{B}}\mu_1 = {}_{\mathcal{G}}\mu_1 \cos^2({}_{\mathcal{G}}\psi_{\text{body}}) + {}_{\mathcal{G}}\mu_2 \sin^2({}_{\mathcal{G}}\psi_{\text{body}}), \\ {}_{\mathcal{B}}\mu_2 = {}_{\mathcal{G}}\mu_3. \end{cases} \tag{14}$$

Finally, we obtain a disturbance-compensated MPPI rollout model by embedding ${}_{\mathcal{B}}\boldsymbol{\mu}$ into the nominal dynamics (1):

$$\begin{cases} {}_{\mathcal{B}}\boldsymbol{x}_{k+1} = {}_{\mathcal{B}}\boldsymbol{x}_k + \mathrm{diag}({}_{\mathcal{B}}\boldsymbol{\mu_k}) \cdot \boldsymbol{R}_{t,k}({}_{\mathcal{B}}\theta) \cdot \boldsymbol{u}_k \cdot T, \\ {}_{\mathcal{B}}\boldsymbol{\mu}_{k+1} = {}_{\mathcal{B}}\boldsymbol{\mu}_k. \end{cases} \tag{15}$$

For clarity, the steps of the proposed FDC-MPPI are summarized in Algorithm 2.

Algorithm 2 Steps of FDC-MPPI Framework

Require: Target goal, ESDF map, odometry data $\boldsymbol{z}_k$, previous control input u^*_{k-1}.
Ensure: Optimal control input u^*_0.

while mission not complete **do**
  // Phase 1: FR-MHE Augmented State Estimation
  Compute gradient of residual cost $\nabla\phi_k(\bar{\boldsymbol{\xi}}_k)$
  Update augmented state estimate:
  $\hat{\boldsymbol{\xi}}_k \leftarrow \bar{\boldsymbol{\xi}}_k - \alpha\nabla\phi_k(\bar{\boldsymbol{\xi}}_k)$
  Extract disturbance coeff. ${}_{\mathcal{G}}\boldsymbol{\mu}$ from $\hat{\boldsymbol{\xi}}_k$
  Propagate prior for next step: $\bar{\boldsymbol{\xi}}_{k+1} \leftarrow f(\hat{\boldsymbol{\xi}}_k, \boldsymbol{u}_{k-1})$
  // Phase 2: Map ${}_{\mathcal{G}}\boldsymbol{\mu}$ to ${}_{\mathcal{B}}\boldsymbol{\mu}$
  ${}_{\mathcal{B}}\mu_1 \leftarrow {}_{\mathcal{G}}\mu_1 \cos^2({}_{\mathcal{G}}\psi_{\text{body}}) + {}_{\mathcal{G}}\mu_2 \sin^2({}_{\mathcal{G}}\psi_{\text{body}})$
  ${}_{\mathcal{B}}\mu_2 \leftarrow {}_{\mathcal{G}}\mu_3$
  // Phase 3: MPPI Control Update
  for $n \leftarrow 0$ to $N-1$ **in parallel do**
    Sampling: $\boldsymbol{u}^n_k \leftarrow \boldsymbol{u}^{\text{nom}}_k + \delta^n_k$
    // Rollout using (15)
    Rollout: ${}_{\mathcal{B}}\boldsymbol{x}_{k+1} \leftarrow \text{Dynamics}({}_{\mathcal{B}}\boldsymbol{x}_k, \boldsymbol{u}_k, {}_{\mathcal{B}}\boldsymbol{\mu})$
    Evaluation: $J_n \leftarrow \sum_{k=0}^{K-1} L_n + \Phi_n$
  end for
  Trajectory weights: $\varepsilon_n \leftarrow \frac{\exp(-(J_n - J_{min})/\lambda)}{\sum_{n=0}^{N-1}\exp(-(J_n - J_{min})/\lambda)}$
  Optimal control: $\boldsymbol{u}^* \leftarrow \boldsymbol{u}^{\text{nom}} + \sum_{n=0}^{N-1}\varepsilon_n\delta\boldsymbol{u}^n$
  Send u^*_0 to low-level controller
  Shift $\boldsymbol{u}^{\text{nom}}$ for next time step
end while

IV. COMPARATIVE EXPERIMENTS

A. Platform and Computation Acceleration

We validate the proposed FDC-MPPI via comparative experiments in both high-fidelity Gazebo and real-world trials on a Unitree GO2 robot. To achieve the high update rates required for reactive obstacle avoidance under severe disturbances, the entire pipeline runs fully onboard an NVIDIA Jetson Orin NX 16GB, and is accelerated with custom CUDA kernels to exploit massive GPU parallelism for rollout evaluation and cost computation [15].

Perception is achieved by a Livox Mid-360 LiDAR that streams real-time point clouds for local mapping. To reduce the overhead of ESDF map construction and distance queries during MPPI rollouts, we leverage the Orin NX shared-memory architecture, allowing both CPU and GPU to access map data directly. This design avoids frequent large host–device transfers of large-scale arrays, which are a common source of latency spikes in embedded GPU deployments, and thereby improves both average runtime and worst-case jitter.

In addition, we adopt a decoupled control–sampling architecture, where the MPPI control update frequency is independent of the sampling interval Δt_{sample}. This decoupling allows the planner to maintain a "far-sighted" local horizon for strategic navigation while reducing total compute consumption per cycle. As a result, the average computation time of a full FDC-MPPI iteration is only 2.42 ms on resource-constrained embedded hardware, enabling stable operation at 150 Hz. Fig. 3 illustrates dynamic obstacle avoidance in a confined environment, where the robot rapidly replans when the obstacle changes position. The main parameters and onboard performance statistics are summarized in TABLE II.

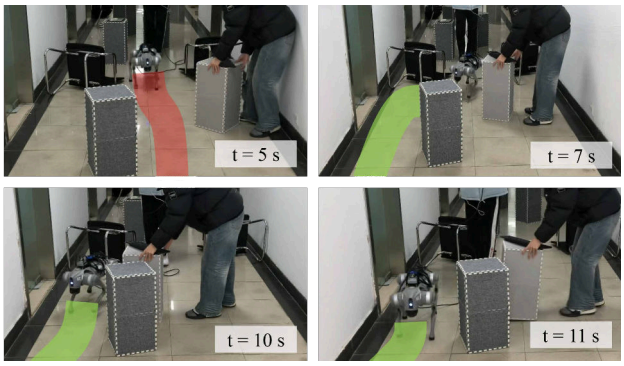


Fig. 3. **Video: FDC-MPPI for dynamic obstacle avoidance.** Red/green indicate motion direction before and after the obstacle moves.

B. Navigation in Cluttered & Disturbed Environments

We task the quadruped with driving through challenging terrains including slippery slopes, outdoor lawn, and also deliberately introduce significant disturbance forces that induce abrupt or irregular variations in the robot's executed velocity. These perturbations produce a pronounced execution mismatch between commanded and realized motion. The proposed FDC-MPPI is compared against the vanilla MPPI, where their definitions are as below for clarity.

(1) **Vanilla MPPI:** MPPI with computation acceleration, without FR-MHE disturbance compensation;
(2) **FDC-MPPI:** MPPI with computation acceleration, while with FR-MHE disturbance compensation.

*1) **Continuous and Slippery Slopes**:* As shown in Fig. 1, after designating target coordinates, the quadrupedal robot was tasked with autonomous obstacle avoidance within a confined space while traversing five consecutive ramps. To induce conditions conducive to slippage and traction loss, the dynamic friction coefficient of selected ramps in the Gazebo simulation was set to 0.2 to simulate slippery surfaces (gray slopes in the first row of images). In physical experiments, smooth Teflon tape was applied to specific ramp surfaces to replicate these low-friction conditions. The criteria for successful obstacle avoidance are defined: the robot must navigate without making contact with any obstacles and successfully reach a position within a 0.5 m radius of the target coordinates.

TABLE II

FDC-MPPI Parameters and Computing Statistics.

MPPI param.				Cost definition constants		Compute performance	
N_{traj}	200	${}^{\mathcal{B}}v_{x\{\max\}}$	0.8 m/s	W_{progress}	80	Control Loop	150 Hz
N_{point}	25	$\omega_{\{\min\}}$	-0.7 rad/s	$W_{\text{collision}}$	45	Min. Latency	1.78 ms
Δt_{sample}	40 ms	$\omega_{\{\max\}}$	0.7 rad/s	$W_{\text{direction}}$	5.0	Max. Latency	4.81 ms
λ	0.12	Σ_v	0.1	$W_{\text{term,residual}}$	10	Aver. Latency	2.42 ms
${}^{\mathcal{B}}v_{x\{\min\}}$	0.15 m/s	Σ_ω	0.9	$W_{\text{term,direction}}$	40	FR-MHE Comp. Time	0.84 ms

* N_{traj} and N_{point} denote No. of sample trajectories and No. of discretization points per trajectory.

As shown in Fig. 4, when robots perform autonomous obstacle avoidance using the vanilla MPPI, they face a high risk of colliding with obstacles or even tipping over, both in Gazebo and real-world trails. The main cause is that disturbance-induced velocity fluctuations and transient tracking lag corrupt the nominal rollout prediction, leading to biased cost evaluation and overly optimistic clearance. Once the rollouts become inaccurate, the importance weights can favor trajectories that appear safe in prediction but are unsafe in execution, producing oscillatory corrections and occasional rollover. In contrast, the proposed FDC-MPPI updates the rollout model via disturbance compensation, providing more stable velocity control and enabling smoother traversal over complex ramps, as displayed in Fig. 1. To evaluate robustness and disturbance rejection, we conducted twenty tests for each algorithm in both Gazebo and physical experiments. The comparative results are reported in TABLE III.

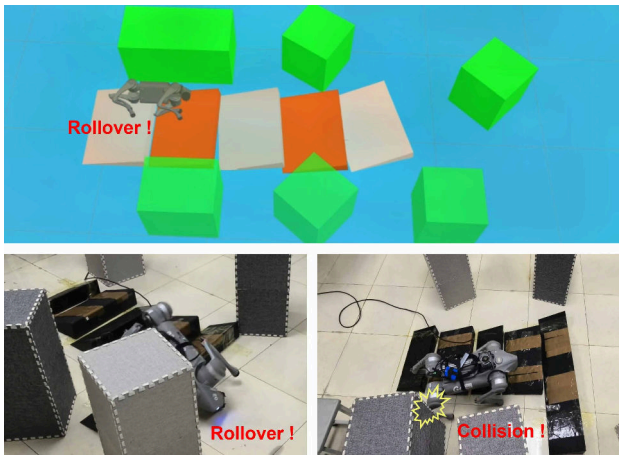


Fig. 4. **Video: Performance of Vanilla MPPI.** It fails to consider the executed velocity under environmental disturbances, making it nearly impossible to smoothly navigate complex terrains.

TABLE III

COMPARATIVE PERFORMANCE STATISTICS

Method	Gazebo	Realworld
Vanilla MPPI	2/20 (10%)	5/20 (25%)
FDC-MPPI	**20/20 (100%)**	**18/20 (90%)**

As illustrated in TABLE III, under the low-friction ramp setting in Gazebo, the Vanilla MPPI completely fails to navigate the prescribed obstacle-cluttered environment. In contrast, FDC-MPPI consistently and robustly completes the avoidance tasks. During physical experiments, despite additional disturbances such as odometry noise, contact variability, and small mapping imperfections, FDC-MPPI achieved a significantly higher success rate, with only two collisions recorded out of 20 trials. Although the Vanilla MPPI successfully completed the task in several instances, its performance remains fragile and inconsistent. As evidenced by the X–Y planar trajectories in Fig. 5, compared to FDC-MPPI, the trajectories generated by the Vanilla MPPI exhibit more pronounced oscillations and maintain a dangerously small clearance from obstacles, which substantially increases the risk of collision.

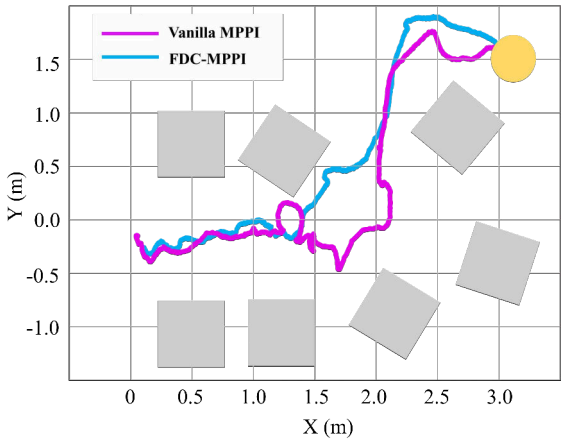


Fig. 5. Trajectories of the robot navigating obstacles in X-Y plane using the Vanilla MPPI (pink path) and FDC-MPPI (blue path).

*2) **Outdoor Lawn**:* Vanilla MPPI fails to account for the tracking performance of the lower-level controller during the sampling process [10]. When traversing slippery, rain-soaked grassland, the robot frequently experiences foot-sinkage and slippage, and the realized motion may lag behind the commanded velocity. These disturbances prevent the MPPI control commands from being accurately executed, leading to significant motion instability, as in Fig. (6b).

Under these conditions, the FR-MHE module estimates ${}_{\mathcal{G}}\boldsymbol{\mu}$ by utilizing the robot's odometry data as the measurement and the previous control input from the FDC-MPPI as the system input. Given that the actual velocity frequently falls short of the commanded velocity due to slippage, this coefficient typically remains below unity. Consequently, the FDC-MPPI generates augmented control commands through

the rollout model (15) that compensate for the velocity loss caused by slippage, thereby maintaining the robot's stability.



(a) Vanilla MPPI (b) FDC-MPPI

Fig. 6. **Video: Performance on Slippery lawn.** FDC-MPPI (b) shows much better stability than the vanilla MPPI (a).

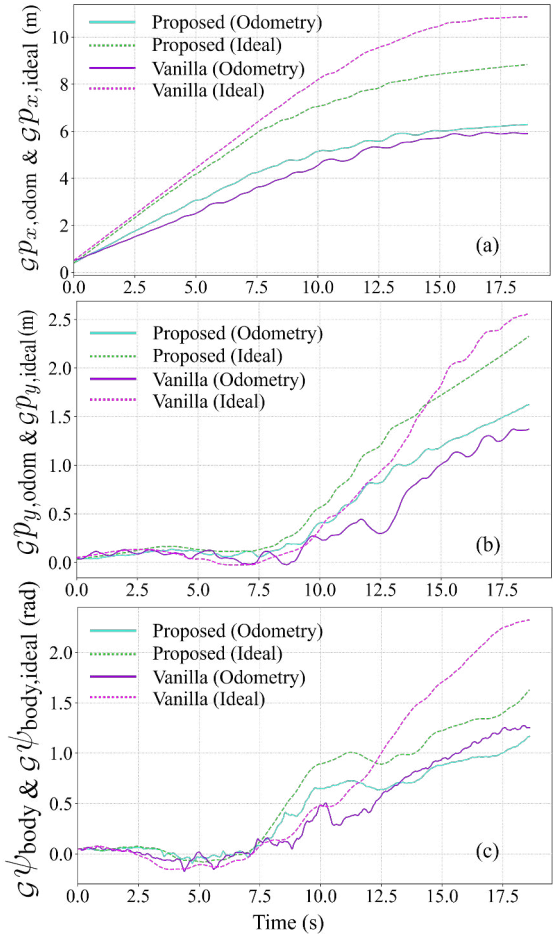


Fig. 7. Comparative tracking performance on slippery lawn.

Fig. 7 presents a comparison between the odometry poses (solid lines) and the ideal trajectories derived from the FDC-MPPI control inputs (dashed lines) in the X-Y plane. It is evident that by augmenting the system model with ${}_{\mathcal{B}}\boldsymbol{\mu}$, the robot's actual motion profile aligns more closely with the reference trajectory. This demonstrates a superior response to high-level velocity commands. Notably, during obstacle avoidance maneuvers using FDC-MPPI, the actual yaw angle in $\{\mathcal{G}\}$ exhibits a trend highly consistent with the ideal state, further validating the tracking performance. In TABLE IV, we compute the RMSE to quantify the closeness between the ideal trajectory and the actual trajectory, and use it to evaluate the robot's execution of the upper-level control quantities computed by FDC-MPPI.

To characterize the oscillation observed in the actual trajectory, the mean curvature is calculated as a geometric smoothness metric. While the Vanilla motion exhibits high-frequency directional shifts due to uncompensated disturbances, the FDC-MPPI maintains a lower average curvature. This reduction signifies that the disturbance-compensated model provides a more consistent execution of the desired path, effectively suppressing parasitic oscillations.

TABLE IV

COMPARATIVE PERFORMANCE METRICS

Method	Clearance (m)	RMSE (m)	Curvature (m^{-1})	
			Avg.	Max.
Vanilla MPPI	0.33±0.09	3.72±0.34	4.91±0.08	41.60±3.45
FDC-MPPI	**0.51±0.07**	**2.64±0.19**	**1.48±0.07**	**12.94±1.13**

* Clearance represents the closest distance to an obstacle along the movement path. A smaller value indicates a higher collision risk.

*3) **Significant External Forces**:* For more intense disturbances, external forces are applied to robot during autonomous obstacle avoidance. As illustrated in Fig. 8, these external forces cause sudden jumps in the robot's displacement and velocity. Thanks to the high-frequency update mechanism of FDC-MPPI, the robot successfully navigates around obstacles even when subjected to abrupt state changes. Fig. 8 depicts the variations of the estimated coefficient ${}_{\mathcal{G}}\boldsymbol{\mu}$ under sudden velocity changes. It follows that FDC-MPPI performs real-time corrections to the rolling optimization model by dynamically updating these coefficients.

V. CONCLUSIONS

We presented FDC-MPPI, a fast and robust autonomous navigation framework for quadrupedal robots in narrow, cluttered and disturbed environments. By integrating a FR-MHE estimation module, the framework maps environmental perturbations into real-time disturbance compensation coefficients, which are then used to rectify the predictive rollout model in real-time. Comparative experimental results from both high-fidelity Gazebo simulations and real-world deployments demonstrate that FDC-MPPI significantly outperforms the Vanilla MPPI in terms of navigation success rate, tracking accuracy, and disturbance rejection. Our system achieves a 150 Hz update frequency on an embedded NVIDIA Jetson Orin NX 16GB, ensuring agile and reactive obstacle avoidance even under sudden external forces or low-friction terrain conditions. Future work will focus on extending this framework to more complex and agile navigation tasks and integrating terrain-aware traversability analysis to further enhance environmental adaptability.

REFERENCES

[1] S. Gangapurwala, L. Campanaro, and I. Havoutis, "Learning low-frequency motion control for robust and dynamic robot locomotion," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5085–5091, IEEE, 2023.

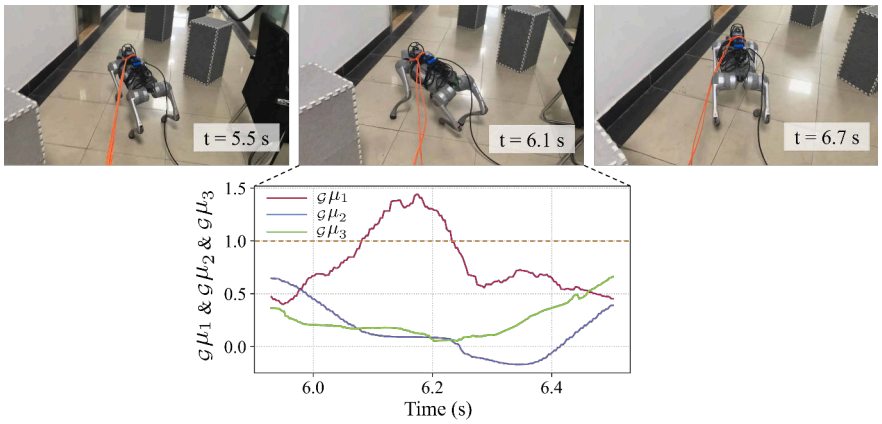


Fig. 8. **Video: FDC-MPPI under violent forces.** The upper illustrates a scenario where manual external forces are applied to the robot during an obstacle avoidance task. The lower characterizes the real-time evolution of disturbance coefficients $_{\mathcal{G}}\boldsymbol{\mu}$. External disturbances cause abrupt changes in the robot's velocity and displacement toward the negative x-axis and positive y-axis in $\{\mathcal{G}\}$, and a sudden increase in angular velocity around the z-axis. Consequently, $_{\mathcal{G}}\mu_1$ exceeds the nominal value of 1 (indicated by the orange dashed line), while $_{\mathcal{G}}\mu_2$ and $_{\mathcal{G}}\mu_3$ fall significantly below their nominal levels.

[2] C. Zhu, Z. Zhang, W. Liu, S. Li, and Y. Liu, "Learning accurate and robust velocity tracking for quadrupedal robots," *Journal of Field Robotics*, vol. 43, no. 1, pp. 76–88, 2026.

[3] J. Queeney, X. Cai, A. Schperberg, R. Corcodel, M. Benosman, and J. P. How, "Gram: Generalization in deep rl with a robust adaptation module," *IEEE Robotics and Automation Letters*, 2025.

[4] H. Cao, J. Shen, Y. Zhang, Z. Fu, C. Liu, S. Sun, and S. Zhao, "Proximal cooperative aerial manipulation with vertically stacked drones," *Nature*, vol. 646, no. 8085, pp. 576–583, 2025.

[5] M. F. AL-Sunni, H. Almubarak, K. Horng, and J. M. Dolan, "Lla-mpc: Fast adaptive control for autonomous racing," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1969–1976, IEEE, 2025.

[6] M. Minařík, R. Pěnička, V. Vonásek, and M. Saska, "Model predictive path integral control for agile unmanned aerial vehicles," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 13144–13151, IEEE, 2024.

[7] H. Zhang, J. Ge, J. Su, Y. Pan, J. Yang, W.-H. Chen, and S. Li, "Pred-mppi: Disturbance-preview and efficient mppi for robust quadrotor tracking with hardware validation," in *2026 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2026.

[8] Z. Huang, R. Li, W. Chen, Z. Lin, Z. Wu, and B. Zhang, "Risk euclidean distance-based model predictive path integral to safety-critical obstacle avoidance," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1906–1913, IEEE, 2025.

[9] P. M. Gupta, O. Procházka, J. Hřebec, M. Novosad, R. Pěnička, and M. Saska, "Lol-nmpc: Low-level dynamics integration in nonlinear model predictive control for unmanned aerial vehicles," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1186–1193, IEEE, 2025.

[10] H. Zhang, A. Srikanthan, S. Folk, V. Kumar, and N. Matni, "Why change your controller when you can change your planner: Drag-aware trajectory generation for quadrotor systems," *ArXiv*, vol. abs/2401.04960, 2024.

[11] M. V. Gasparino, A. N. Sivakumar, Y. Liu, A. E. Velasquez, V. A. Higuti, J. Rogers, H. Tran, and G. Chowdhary, "Wayfast: Navigation with predictive traversability in the field," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10651–10658, 2022.

[12] J. Lu, J. Su, W.-H. Chen, and S. Li, "Fr-mhe: Fast and robust quadrotor state estimation with design, analysis and validation," *IEEE Transactions on Industrial Electronics*, 2026.

[13] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *2016 IEEE international conference on robotics and automation (ICRA)*, pp. 1433–1440, IEEE, 2016.

[14] J. Alvarez-Padilla, J. Z. Zhang, S. Kwok, J. M. Dolan, and Z. Manchester, "Real-time whole-body control of legged robots with model-predictive path integral control," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 14721–14727, IEEE, 2025.

[15] G. Turrisi, V. Modugno, L. Amatucci, D. Kanoulas, and C. Semini, "On the benefits of gpu sample-based stochastic predictive controllers for legged locomotion," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 13757–13764, IEEE, 2024.

[16] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, "Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4974–4981, IEEE, 2025.

[17] J. R. Han, H. Thomas, J. Zhang, N. Rhinehart, and T. D. Barfoot, "Dr-mpc: Deep residual model predictive control for real-world social navigation," *IEEE Robotics and Automation Letters*, 2025.

[18] H. Li, Q. Zhang, and D. Zhao, "Deep reinforcement learning-based automatic exploration for navigation in unknown environment," *IEEE transactions on neural networks and learning systems*, vol. 31, no. 6, pp. 2064–2076, 2019.

[19] L. Wellhausen and M. Hutter, "Rough terrain navigation for legged robots using reachability planning and template learning," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 6914–6921, IEEE, 2021.

[20] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, "Dd-ppo: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *International Conference on Learning Representations*.

[21] H. Zhang, J. Ge, J. Su, K. Gu, F. Wang, W.-H. Chen, and S. Li, "Model predictive control with residual learning and real-time disturbance rejection: Design and experimentation," *Control Engineering Practice*, vol. 165, p. 106587, 2025.

[22] L. Zou, Z. Wang, J. Hu, and Q.-L. Han, "Moving horizon estimation meets multi-sensor information fusion: Development, opportunities and challenges," *Information Fusion*, vol. 60, pp. 1–10, 2020.

[23] W. Choo and E. Kayacan, "Data-based mhe for agile quadrotor flight," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4307–4314, IEEE, 2023.

[24] C. Meng and H. Li, "Learning-based nonlinear mpc with integrated moving horizon estimation of quadrotors," *IEEE Transactions on Industrial Electronics*, 2025.

[25] W. Li, J. Su, C. Liu, W.-H. Chen, and S. Li, "Dr-mpc: Disturbance-resilient model predictive visual servoing control for quadrotor uav pipeline inspect," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 20738–20745, IEEE, 2025.

[26] Z. Yang, S. Wang, T. Zhou, and F. Wang, "Efficient trajectory optimization for car-like robots with esdf-driven soft constraints," in *2026 38th Chinese Control and Decision Conference (CCDC)*, pp. 6166–6171, 2026.

[27] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, vol. 8, no. 82, p. eadg1462, 2023.